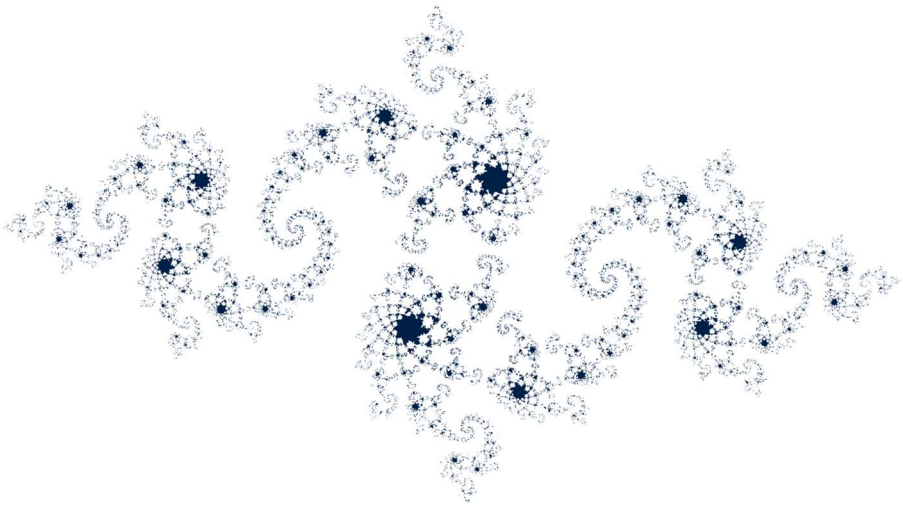
Prueba Técnica

Kapital Grupo Financiero

Emiliano Fuentes León



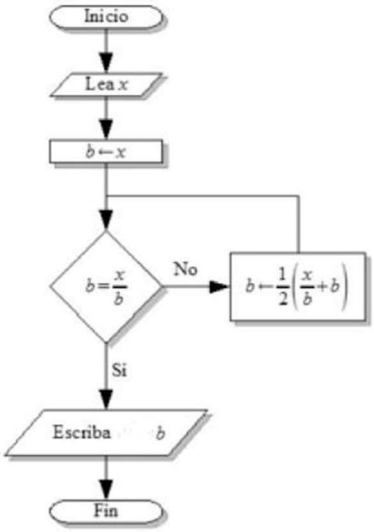
Repositorio:

https://github.com/LordChinos/An-lisis_Riesgo_Crediticio_ML.git

Primera sección.

Problemas:

1. Se cuenta con el siguiente diagrama de flujo



a) Escribe el pseudo código del diagrama.
b) ¿Qué función realiza el programa?
c) Describa la complejidad del algoritmo (O(g(n))) que se implementó.

Solución.

a) Pseudo código

```
Algoritmo

    Leer x
    b <- x

    Mientras (b != x/b) hacer
        b <- ((x/b)+b)/2

    Escribir b

Fin_Algoritmo
```

b) Este algoritmo corresponde con el algoritmo de Newton para encontrar la raíz cuadrada de un número real dado, sin embargo, el

algoritmo por cómo fue presentado tiene limitantes, únicamente converge para valores de 'x' positivos, y debido a los errores de cálculos y limitantes de hardware, la condición de igualdad estricta puede generar un bucle infinito, se debe tener en cuenta un valor de tolerancia para que el programa funcione sin problemas.

Para x=0 bastaría con hacer b <- x y regresar b.

c) La complejidad del algoritmo es de **O(1)**, esto porque el método de Newton para la raíz cuadrada tiene una aproximación cuadrática, esto es, los errores de cada iteración:

$$e_{n+1} \propto e_n^2$$

Entonces se puede deducir que para cuando el error es tan pequeño que tras n iteraciones es menor al épsilon de la máquina:

$$n \approx \log(\log(1/\varepsilon))$$

Entonces de aquí se deduce que, debido a que el épsilon de la máquina es constante el orden **O(log(log(1/epsilon)))=O(1)**.

Únicamente en el caso de que se pudiera ajustar el valor de tolerancia épsilon, se tendría:

$$\mathrm{O}(\log(\log(1/\varepsilon)))$$

2. Escribe un pseudo código recursivo y otro iterativo para calcular el factorial de un número dado.

a) Describa las diferencias

b) ¿Cuál de ellos es más eficiente y por qué?

Solución.

```
Función_Factorial(n)

    Si (n == 0) o (n == 1):
        Regresar 1
    Si no:
        Regresar n*Función_Factorial (n-1)

Fin_Función
```

```
Algoritmo

    Leer n
    Si (n < 0):
        Escribir "Factorial no valido para negativos"
    Si no:
        Factorial <- Función_Factorial(n)
    Escribir Factorial

Fin_Algoritmo
```

Algoritmo iterativo:

```
Algoritmo

    Factorial <- 1
    Leer n

    Si (n<0):
        Escribir "Factorial no valido para negativos"
    Si (n == 0):
        Escribir Factorial
    Si no:
        Para i <- 1 hasta n hacer:
            Factorial <- Factorial*i
        Escribir Factorial
Fin_Algoritmo
```

a) EL algoritmo iterativo, a diferencia del recursivo, hace uso un bucle bien definido, es decir, el algoritmo conoce el límite de veces que se iterará la función dentro del bucle.
El algoritmo iterativo no hace uso de bucles, la función se llama a sí misma, guardando memoria en la computadora. El algoritmo en general no sabe cuándo se detendrá, ni cuantas llamadas realizará.

b) El más eficiente es el **iterativo**, ambos son algoritmos de complejidad **O(n)**, sin embargo, el iterativo consume menos memoria, todos los cálculos se guardan y actualizan en únicamente una variable, más el contador, por lo que su complejidad espacial es de **O(1)**, por su

parte el algoritmo recursivo, genera una pila de llamadas de tamaño n, por lo que en cuanto memoria, su complejidad es **O(n)**.

3. Sea T el conjunto de registros de la tabla, donde cada registro es una tupla con los elementos (Nombre, Fecha de nacimiento, RFC). Por tanto,

$$T=\{ (NOMBRE_1, FECHA_1, RFC_1), (NOMBRE_2, FECHA_2, RFC_2), \ldots, (NOMBRE_{10,000,000}, FECHA_{10,000,000}, RFC_{10,000,000})\}$$

Dado que la tabla contiene registros repetidos, definimos el conjunto R como el subconjunto de registros duplicados en T . Tal que,

$$R=\{(NOMBRE_i, FECHA_i, RFC_i) \mid \text{al menos existe otra tupla con los mismos elementos}\}$$

Ahora definimos otro subconjunto de T. Sea,

$$U=\{(NOMBRE_i, FECHA_i, RFC_i) \mid \text{no hay otra tupla con los mismos elementos}\}$$

a) Escriba un script de SQL que muestre al subconjunto R de T.
b) Escriba un script de SQL que muestre al subconjunto U de T.

Solución.

a) Subconjunto R:

```
SELECT * FROM users GROUP BY
    NOMBRE,
    FECHA,
    RFC
HAVING COUNT(*)>1
```

b) Subconjunto U:

```
SELECT * FROM users GROUP BY
    NOMBRE,
    FECHA,
    RFC
```

```
HAVING COUNT(*)=1
```

Segunda sección.

Introducción.

El otorgamiento de crédito es un proceso mediante el cual una institución financiera, evalúa al solicitante para detectar si concederle el préstamo, crédito o financiamiento es viable, y en qué condiciones se le puede otorgar, en estas condiciones se puede inclusive ajustar la cantidad monetaria que se le otorgará, siendo menor o mayor a la solicitada.

Para otorgar un crédito, regularmente se busca que la institución financiera adquiera cierto beneficio del préstamo ya sea mediante intereses o alguna comisión tras el pago del crédito (esto en general no tiene que ser siempre lo que se busca, se pueden acceder a créditos sin intereses y sin pagos extras, eso depende de la institución y de la situación), y también se busca minimizar el riesgo de adeudos.

Las fases se pueden resumir en 6:

1. **Solicitud.** El solicitante debe presentarse ante la institución junto sus documentos cómo lo son en caso de una personal moral
    - INE o alguna identificación oficial.
    - RFC y constancia de situación fiscal.
    - Comprobante de domicilio.
    - Estados financieros:

2. **Verificación de identidad y cumplimiento.** A partir de esta fase, la institución es quien lleva a cabo el proceso. Cómo indica el nombre, se verifica la identidad de la persona y se revisa la situación fiscal de la persona, si esta tiene deudas o si pertenece a buró de crédito, entre otros.

3. **Análisis crediticio.** En esta etapa se analiza la situación económica de la persona y otras características que puedan medir el riesgo de adeudo. Todo este análisis se hace tomando en cuenta características cuantitativas y cualitativas cómo lo son, ingresos, cantidad solicitada, edad, estado civil, sexo, etc…

4. **Dictamen.** Esta parte dependiendo de los resultados de la fase anterior, puede ser la que finalice el proceso negando al solicitante el préstamo, o por otra parte abre pase a la siguiente fase en caso de que sea aprobado el préstamo y en qué condiciones se realizará.

5. **Formalización.** Es la fase en la que se hace un contrato para que sea firmado por el solicitante en caso de estar de acuerdo con las condiciones establecidas. En caso de que el solicitante firme, se inicia con el registro de la persona ante las autoridades correspondientes, principalmente para que sea oficial que se realizará el préstamo.

6. **Seguimiento.** Una vez realizado el préstamo, la institución financiera le da seguimiento al deudor para conocer su situación financiera (esto para asegurarse que el deudor tenga la capacidad de poder pagar conforme va pasando el tiempo). También se realiza un monitoreo de cómo se va cubriendo el préstamo.

Planteamiento del problema.

La tabla de datos contiene la información real de 1000 clientes que solicitaron un crédito en Alemania, clasificada en 20 atributos, en esta tabla se incluye una última columna, en donde se clasificó si el cliente resultó alguien que cumplió con la deuda, con el dato **"1=Bueno";** y quienes tuvieron serios problemas al pagar la deuda o que inclusive no la pagaron, con el dato **"2=Malo"**.

En la siguiente tabla se muestran los atributos:

	Tipo de dato	Qué es	Qué datos hay
Atributo 1	Cualitativo	Cuenta corriente	▪ A11: …< 0 DM ▪ A12: 0 <= … < 200 DM

			▪ A13: 200 DM <= … ▪ A14: Sin cuenta
Atributo 2	Cuantitativo	Duración en meses	---
Atributo 3	Cualitatiov	Historial crediticio	▪ A30: Sin créditos/Créditos pagados a tiempo ▪ A31: Créditos en este banco pagados a tiempo ▪ A32: Créditos existentes bien devueltos hasta ahora ▪ A33: Retraso en el pago en el pasado ▪ A34: Cuenta crítica/Créditos existentes en otros bancos
Atributo 4	Cualitativo	Propósito	▪ A40: Carro nuevo ▪ A41: Carro usado ▪ A42: Muebles ▪ A43: Radio/Televisión ▪ A44: Electrodomésticos ▪ A45: Reparaciones ▪ A46: Educación ▪ A47: Vacaciones ▪ A48: Reentrenamiento ▪ A49: Negocios ▪ A410: Otros
Atributo 5	Cuantitativo	Monto de crédito	---
Atributo 6	Cualitativo	Ahorros	▪ A61: … < 100 DM ▪ A62: 100 <= … < 500 DM ▪ A63: 500 <= … < 1000 DM ▪ A64: 1000 DM <= …
Atributo 7	Cualitativo	Tiempo en el actual empleo	▪ A71: Desempleado ▪ A72: … < 1 año ▪ A73: 1 <= … < 4 años ▪ A74: 4 <=… < 7 años ▪ A75: 7 años <= …
Atributo 8	Cuantitativo	Porcentaje del ingreso disponible destinado al pago del crédito	---
Atributo 9	Cualitativo	Sexo y estado civil	▪ A91: Hombre – Divorciado/Separado ▪ A92: Mujer – Divorciada/Separada/Casada ▪ A93: Hombre – Soltero

			▪ A94: Hombre – Casado/Viudo ▪ A95: Mujer - Soltera
Atributo 10	Cualitativos	Otros deudores/Garantes	▪ A101: Ninguno ▪ A102: Co-solicitante ▪ A103: Garante
Atributo 11	Cuantitativo	Tiempo en residencia actual (Años)	---
Atributo 12	Cualitativo	Propiedad	▪ A121: Bienes Raices ▪ A122: Ahorro sociedad de construcción/Seguro de vida ▪ A123: Carro u otros ▪ A124: Desconocido/Sin Propiedades
Atributo 13	Cuantitativo	Edad (Años)	---
Atributo 14	Cualitativo	Otros planes de pago	▪ A141: Banco ▪ A142: Tiendas ▪ A143: Ninguno
Atributo 15	Cualitativo	Hogar	▪ Renta ▪ Propio ▪ Gratuito
Atributo 16	Cuantitativo	Cantidad de créditos existentes en este banco	…
Atributo 17	Cualitativo	Trabajo	▪ A171: Desempleado/No calificado - No es residente ▪ A172: No calificado – Residente ▪ A173: Empleado calificado ▪ A174: Empleado muy calificado/Auto empleado
Atributo 18	Cuantitativo	Personas dependientes del solicitante	---
Atributo 19	Cualitativo	Teléfono	▪ A191: No ▪ A192: Sí
Atributo 20	Cualitativo	Trabajador extranjero	▪ A201: Sí ▪ A202: No

Además, la tabla no cuenta con ningún dato sin llenar, todas las columnas están llenas.

Desarrollo.

Reducción de dimensionalidad.

Se puede realizar una reducción de dimensionalidad con el método PCA, para esto, se usó la librería Scikit-learn. Usando el código

```python
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
import pandas as pd
import seaborn as sb

nombres=[x for x in range(1,25)]+['Resultado']
datos = pd.read_csv('german.data-numeric', sep='\s+', header=None, names=nombres)
```

[1] ✓ 2.6s Python

```python
print(datos)

# Captura de datos tomando, quitando la columna de resultados
X=datos.drop(labels=datos.columns[-1], axis=1)
```

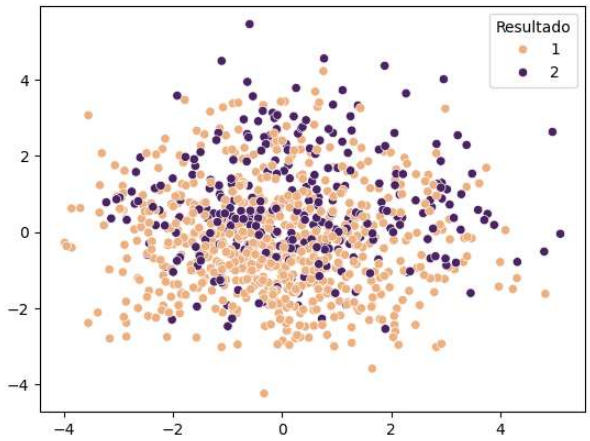
[2] ✓ 0.1s Python

```
...      1   2  3   4  5  6  7  8  9  10  ...  16  17  18  19  20  21  22  23  24  \
0     1   6  4  12  5  5  3  4  1  67  ...   0   0   1   0   0   1   0   0   1
1     2  48  2  60  1  3  2  2  1  22  ...   0   0   1   0   0   1   0   0   1
2     4  12  4  21  1  4  3  3  1  49  ...   0   0   1   0   0   1   0   1   0
3     1  42  2  79  1  4  3  4  2  45  ...   0   0   0   0   0   0   0   0   1
4     1  24  3  49  1  3  3  4  4  53  ...   1   0   1   0   0   0   0   0   1
..   ..  ..  ..  ..  ..  ..  ..  ..  ..  ..  ...  ..  ..  ..  ..  ..  ..  ..  ..  ..
995   4  12  2  17  1  4  2  4  1  31  ...   0   0   1   0   0   1   0   1   0
996   1  30  2  39  1  3  1  4  2  40  ...   0   1   1   0   0   1   0   0   0
997   4  12  2   8  1  5  3  4  3  38  ...   0   0   1   0   0   1   0   0   1
998   1  45  2  18  1  3  3  4  4  23  ...   0   0   1   0   0   0   0   0   1
999   2  45  4  46  2  1  3  4  3  27  ...   0   1   1   0   0   1   0   0   1

     Resultado
0            1
1            2
2            1
3            1
4            2
..         ...
995          1
996          1
997          1
998          2
999          1

[1000 rows x 25 columns]
```

```python
escal = StandardScaler()
X_escalados = escal.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_escalados)

# Varianza
print(f"Varianza capturada por 2 componentes: {pca.explained_variance_ratio_.sum():.2%}")

#Gráfica
sb.scatterplot(x=X_pca[:,0],y=X_pca[:,1],hue=datos['Resultado'], palette='flare')
```

✓ 0.7s Python

```
Varianza capturada por 2 componentes: 19.33%
```



Como se observa, se usó la tabla de datos **"german.data-numeric"** en lugar de la tabla de datos base, los datos que se manejan son los mismos, sin embargo, la tabla data-numeric facilita la manipulación de datos para librerías cómo Scikit-learn, ahorrando el convertir los datos categóricos en datos numéricos, cabe mencionar, que el procedimiento es sencillo, únicamente bastaría usar métodos cómo **"One-Hot Encoding"** o **" Label Encoding"**. Cómo se puede observar, se obtuvo una varianza del **19.33%.** Puede que parezca un dato bajo, y si bien, no es una varianza muy precisa, hay que considerar que se redujo una base de datos de 20 atributos en únicamente 2.

Comparativo con dos modelos de aprendizaje no supervisado.

Para esta parte se elegirán los modelos:

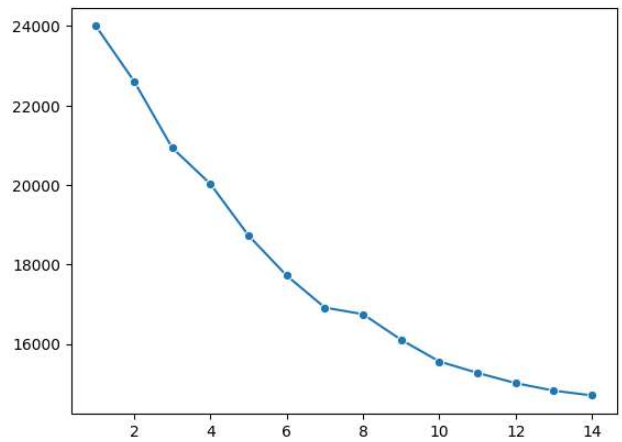
- **K-Mean.** Genera clases de equivalencia en los datos, mediante su proximidad a los centroides que se generan, la cantidad de clases se puede elegir, existen diversos métodos, aquí se decidió usar el método del **codo**, consiste en graficar las inercias en función de la cantidad de cluster/clases generadas, y elegir el valor en donde se vea un "codo" pronunciado.
- **Isolation Forest.** Clasifica los datos en dos tipos, normales =1 y anómalos = -1, para esto compara los atributos de todos los datos, y para saber hasta qué punto algo se considera normal o anómalo se debe dar un parámetro llamado "contaminación". Para obtener este dato, se debe calcular mediante un isolation forest con una contaminación básciamente arbitraria, por lo regular se usa "auto" que es la que trae por defercto SciKit-Learn, y graficando los Scores (qué tan anómalos son los datos entre 0 y 1), ascendentemente, en el punto del eje x en donde se observe un cambio bruzco en la gráfica, se determina el porcentaje de anormalidad/contaminación.

K-Mean.

Para este modelo se usó el siguiente código:

```python
from sklearn.cluster import KMeans

inercia=[]
for k in range(1,15):
    kmeans = KMeans(n_clusters=k, random_state=777)
    kmeans.fit(X_escalados)
    inercia.append(kmeans.inertia_)

sb.lineplot(x=range(1,15), y=inercia, marker='o')
```

[22] ✓ 1.1s Python



En esta gráfica se muestra la **Inercia** de cada cluster, la Inercia se define de la forma:

$$I = \sum_{i=1}^{k} \sum_{x \in C_i} \|x - \mu_i\|^2$$

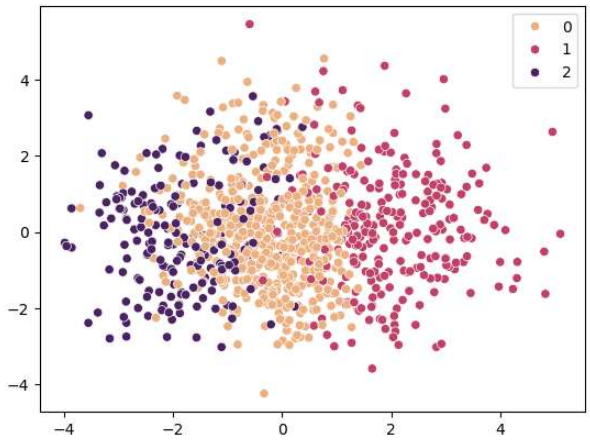
En donde k es la cantidad de clusters, C_i es el i-ésimo cluster y μ_i es su centroide.

Por le método del **codazo** el cual dice que para decidir el valor de k, hay que observar en donde la gráfica forma un "codo", se pueden observar dos valores muy buenos **k=3,7**. Sin embargo, la decisión será **k=3**, esto es porque 7 es una cantidad considerablemente elevada que no permitirá diferenciar tan sencillamente a aquellos elementos de distitnso clusters.

Mediante PCA se puede graficar los datos, segmentándolos según sus clusters.

```
kmeans = KMeans(n_clusters=3, random_state=777)
k_clases = kmeans.fit_predict(X_escalados)

sb.scatterplot(x=X_pca[:,0],y=X_pca[:,1],hue=k_clases, palette='flare')
```

[6] ✓ 0.7s Python



Cómo se puede ver, proyectadas las componentes en 2 dimensiones, se puede observar que las clases generadas por K-mean se pueden diferenciar considerablemente.

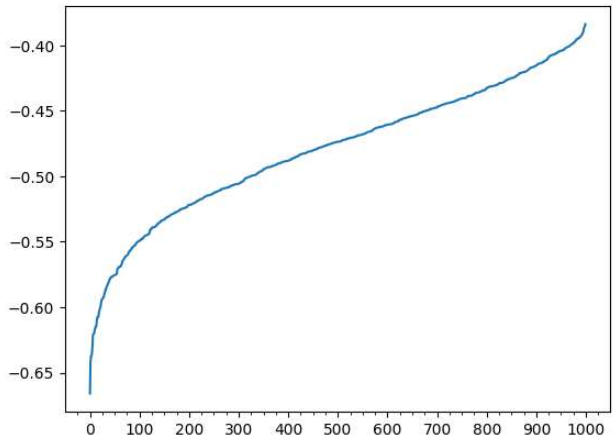
Isolation Forest.

De igual forma se utilizó la librería Scikit-Learn, para programar el isolation forest, primero se ajusta con una contaminación "auto", para verificar qué contaminación será la más idónea. La cantidad de estimadores o árboles será el que la librería trae por defecto, 100 árboles.

Hay que recordar que el método de isolation forest lo que hace es detectar datos que difieren considerablemente del resto de datos, es decir, detecta anomalías.

```python
from sklearn.ensemble import IsolationForest
import matplotlib.pyplot as plt
import matplotlib.ticker as ticker

isoforest=IsolationForest(contamination="auto", random_state=777)
clases_isoforest=isoforest.fit_predict(X)

aux=isoforest.score_samples(X)
aux.sort()
ejes= sb.lineplot(x=range(len(aux)),y=aux, markers="o",)
ejes.xaxis.set_major_locator(ticker.MultipleLocator(100))
ejes.xaxis.set_minor_locator(ticker.MultipleLocator(25))
plt.show()
```

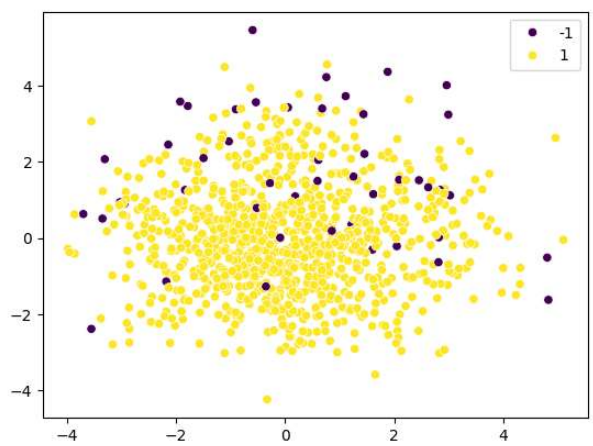
[27] ✓ 0.2s Python



Se observa de la gráfica que hay una pendiente muy pronunciada para datos entre 50 y 75, por lo que se decide tomar una contaminación de 0.05 (5%).

```python
isoforest=IsolationForest(contamination=0.05,random_state=777)
clases_isoforest=isoforest.fit_predict(X)

sb.scatterplot(x=X_pca[:,0],y=X_pca[:,1],hue=clases_isoforest, palette="viridis")
```

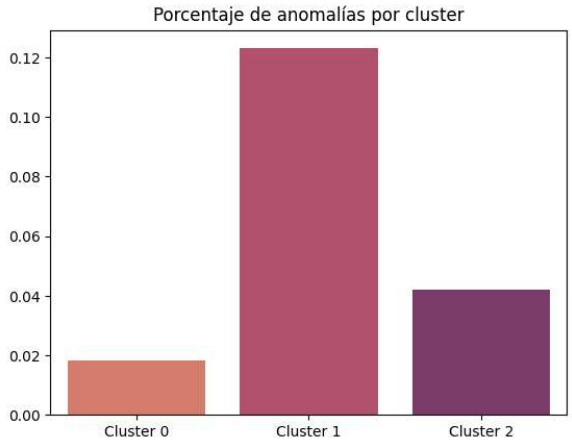
[32] ✓ 0.2s Python



Ahora con estos dos modelos, podemos comparar resultados para que nos indique cómo se relacionan. Se puede detectar la probabilidad de que si se elige un elemento de algún cluster este resulte una anomalía, se usa el código:

```
porcentaje_kmeans_isolation=[0,0,0]

no_supervisado=pd.DataFrame(
    {
        "K_Means": k_clases,
        "Isolatio_Forest": clases_isoforest
    }
)

interseccion = no_supervisado[no_supervisado['Isolatio_Forest'] == -1]

porcentaje_kmeans_isolation[0]=(interseccion['K_Means']==0).sum()/((k_clases==0).sum())
porcentaje_kmeans_isolation[1]=(interseccion['K_Means']==1).sum()/((k_clases==1).sum())
porcentaje_kmeans_isolation[2]=(interseccion['K_Means']==2).sum()/((k_clases==2).sum())

tabla=sb.barplot(x=["Cluster 0","Cluster 1","Cluster 2"],y=porcentaje_kmeans_isolation, palette="flare"
tabla.set(title="Porcentaje de anomalías por cluster")
plt.show()
```

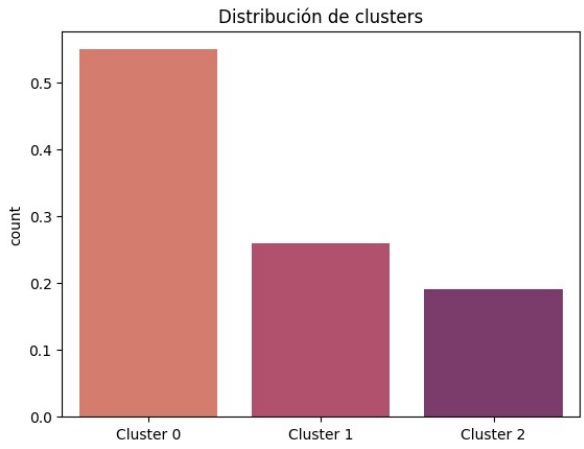
[79] ✓ 0.1s Python



Cómo se puede ver, el mayor porcentaje de anomalía surge en el cluster 1, y las probabilidades son considerablemente distintas cómo apara considerar que las anomalías se distribuyen de igual forma.

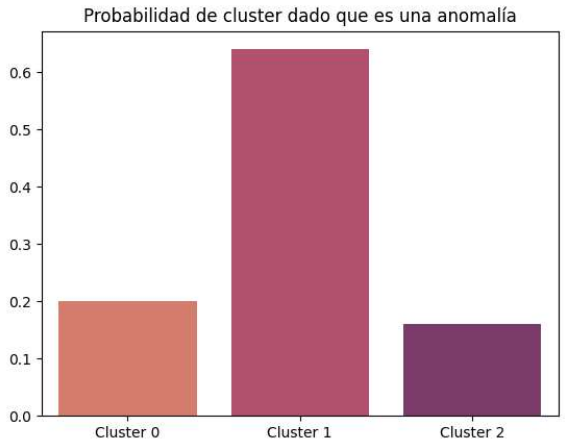
Y más aún, si se considera la proporción que tiene cada cluster, cómo se obtuvo mediante el código:

```
proba_cluster=no_supervisado["K_Means"].value_counts()/len(no_supervisado)
proba_anomalia=(no_supervisado["Isolatio_Forest"]==-1).sum()/len(no_supervisado)

tabla=sb.barplot(x=["Cluster 0","Cluster 1","Cluster 2"],y=proba_cluster, palette="flare")
tabla.set(title="Distribución de clusters")
plt.show()
[80]  ✓ 0.2s                                                                    Python
```



Se observa que el Cluster 0 es el que tiene mayor presencia en los datos, y aún así es el que tiene menos anomalías, esto es sinónimo de que aquellos valores que pertenezcan al cluster 0 son en general los datos más "confiable". También se puede calcular mediante el **Teorema de Bayes**, la probabilidad de que, dada una anomalía, esta tenga pertenezca a alguno de los clusters.

```
probabilidad_bayes=[0,0,0]

probabilidad_bayes[0]=(porcentaje_kmeans_isolation[0]*proba_cluster[0])/proba_anomalia
probabilidad_bayes[1]=(porcentaje_kmeans_isolation[1]*proba_cluster[1])/proba_anomalia
probabilidad_bayes[2]=(porcentaje_kmeans_isolation[2]*proba_cluster[2])/proba_anomalia

tabla=sb.barplot(x=["Cluster 0","Cluster 1","Cluster 2"],y=probabilidad_bayes, palette="flare")
tabla.set(title="Probabilidad de cluster dado que es una anomalía")
plt.show()
[81]  ✓ 0.2s                                                                    Python
```



De aquí nuevamente se observa lo que dio la primera gráfica de barras. El cluster 1 es aquel que contiene el perfil de los datos más propensos a ser anómalos, es decir, se puede crear una tabla que indique el nivel de que un dato sea propenso a ser anómalo en función del cluster al que pertenezca:

Cluster	Qué tan propenso es a ser anómalo
0	Poco propenso
1	Muy propenso
2	Probabilidad media

Comparativo con dos modelos de aprendizaje supervisado.

Los modelos de aprendizaje que se utilizarán serán:

- **Random forest.** Este modelo al igual que Isolation forest, trabaja con árboles de decisiones, sin embargo, estos a diferencia de isolation Forest, buscan tomando en cuenta una muestra de los datos de la columna respuesta y de los datos del resto de la tabla, los árboles se se encuentran en paralelo y el resultado se toma teniendo en cuenta los resultados de los árboles.
- **Gradient Boosting.** También trabaja con árboles, sin embargo, a diferencia de Random Forest, los árboles están en cadena, donde un árbol corrige los errores cometidos por el anterior.

Si hay underfiting se verificará comparando la entrada muestra con la predicción, y si hay overfiting se verificará comparando la predicción de

otra muestra de datos Y_2 con la predicción acorde a los datos entrada de la muestra Y_2.

Random Forest.

Primero se dividen los datos en una proporción de 75%-25%, para tener el 75% de los datos cómo datos de entrenamiento y el resto de prueba, después con SciKit-Learn se usan los parámetros usuales de Scikit-Learn, para verificar si estos son lo suficientemente buenos para el random forest.

```python
from sklearn.model_selection import train_test_split

X=datos.drop("Resultado", axis=1)
Y=datos["Resultado"]

X.shape, Y.shape

X_train, X_test, Y_train, Y_test = train_test_split(X,Y, test_size=0.25, random_state=777)
```

[11] ✓ 0.0s Python

```python
from sklearn.ensemble import RandomForestClassifier

ranForest=RandomForestClassifier(random_state=777, max_depth=10)
ranForest.fit(X_train, Y_train)
```

[15] ✓ 0.1s Python

```
RandomForestClassifier
▸ Parameters
```

```python
from sklearn.metrics import accuracy_score


Z=ranForest.predict(X_train)
print(f"Precisión en entrenamiento: {accuracy_score(Y_train,Z):.2%}")
Z=ranForest.predict(X_test)
print(f"Precisión en prueba: {accuracy_score(Y_test,Z):.2%}")
```

[16] ✓ 0.0s Python

```
Precisión en entrenamiento: 97.33%
Precisión en prueba: 77.20%
```

Cómo se aprecia, existe una precisión de 97.33% en los datos de entrenamientos con los parámetros por defecto, sin embargo, al aplicar la predicción a los datos de prueba se observa una precisión del **77.2%**, hay una diferencia de alrededor del 20%, lo cual es símbolo de **overfiting**.

Entonces se ajustaron los parámetros para evitar este problema.

```python
rest=RandomForestClassifier(random_state=777, max_depth=8, min_samples_split=5, min_samples_leaf=5)
rest.fit(X_train, Y_train)
```

[18] ✓ 0.1s Python

```
RandomForestClassifier
▸ Parameters
```

```python
Z=ranForest.predict(X_train)
print(f"Precisión en entrenamiento: {accuracy_score(Y_train,Z):.2%}")
Z=ranForest.predict(X_test)
print(f"Precisión en prueba: {accuracy_score(Y_test,Z):.2%}")
```

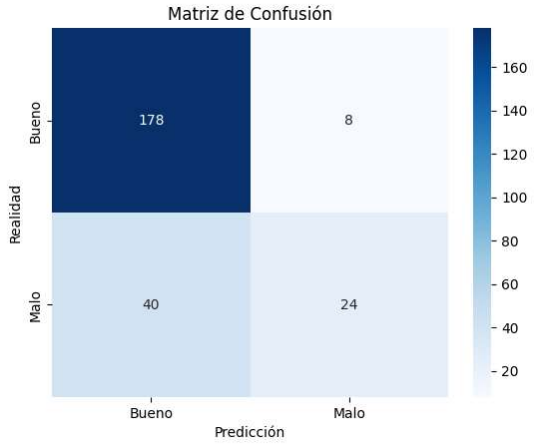
[19] ✓ 0.0s Python

```
Precisión en entrenamiento: 85.60%
Precisión en prueba: 80.80%
```

Cómo se aprecia se tiene ahora una precisión en los datos de entrenamiento del 85.6% y en los de prueba una precisión de 80.8%, menos del 5% de diferencia, y los porcentajes son bastante elevados, es decir, el árbol de decisión tiene es muy preciso, para medir esta precisión se usará la matriz de confusión.

```python
from sklearn.metrics import confusion_matrix

mtrx=confusion_matrix(Y_test,Z)

sb.heatmap(mtrx, annot=True, fmt="d", cmap="Blues",
           xticklabels=['Bueno', 'Malo'],
           yticklabels=['Bueno', 'Malo'])
plt.xlabel('Predicción')
plt.ylabel('Realidad')
plt.title('Matriz de Confusión')
plt.show()
```

[31] ✓ 0.3s Python



Se puede observar de la matriz que el random rorest calcularon los valores que se muestran, donde se considera a Positivo=1 y Negativo =2.

```python
print(mtrx)

VP=mtrx[0,0]
FP=mtrx[1,0]
VN=mtrx[1,1]
FN=mtrx[0,1]

Exactitud=(VP+VN)/mtrx.sum()
print(f"Exactitud: {Exactitud:.2%}")
Prevalencia=(VP+FP)/mtrx.sum()
print(f"Prevalencia: {Prevalencia:.2%}")
Sensibilidad=VP/(VP+FN)
print(f"Sensibilidad: {Sensibilidad:.2%}")
Especificidad=VN/(VN+FP)
print(f"Especificidad: {Especificidad:.2%}")
Precision=VP/(VP+FP)
print(f"Precision: {Precision:.2%}")
```

[71] ✓ 0.0s Python

```
[[178   8]
 [ 40  24]]
Exactitud: 80.80%
Prevalencia: 87.20%
Sensibilidad: 95.70%
Especificidad: 37.50%
Precision: 81.65%
```

	Procentajes
Exactitud	80.80%
Prevalencia	87.20%
Sensibilidad	95.70%
Especificidad	37.50%
Precisión	81.65%

Cómo se ve, el modelo es muy bueno al momento de detectar buenos y malos de forma general, sin embargo, su mayor problema es la proporción entre Malos detectados cómo Malos y los Malos detectados cómo Buenos, el porcentaje es muy bajo, eso quiere decir que el modelo es muy ciego ante el riesgo y no tanto ante los que no representan un riesgo.

Gradient Boosting.

Al igual que en Random Forest, se va a comenzar con los parámetros por defecto que tiene y se verificará si tiene overfiting o underfiting

```python
from sklearn.ensemble import GradientBoostingClassifier

gradBoost=GradientBoostingClassifier(random_state=777)
gradBoost.fit(X_train, Y_train)
```

[73] ✓ 0.4s Python

```
GradientBoostingClassifier
▸ Parameters
```

```python
Z=gradBoost.predict(X_train)
print(f"Precisión en entrenamiento: {accuracy_score(Y_train,Z):.2%}")
Z=gradBoost.predict(X_test)
print(f"Precisión en prueba: {accuracy_score(Y_train,Z):.2%}")
```

[74] ✓ 0.0s Python

Como se aprecia, al igual que en random forest, se presenta overfiting con los atributos por defecto de la librería. Entonces se realiza un cambio de parámetros para eliminar el overfiting.

```python
ostingClassifier(random_state=777, max_depth=5, min_samples_split=3, subsample=0.7, learning_rate=0.01)
n, Y_train)
```

[87] ✓ 0.2s Python

```
GradientBoostingClassifier
▸ Parameters
```

```python
Z=gradBoost.predict(X_train)
print(f"Precisión en entrenamiento: {accuracy_score(Y_train,Z):.2%}")
Z=gradBoost.predict(X_test)
print(f"Precisión en prueba: {accuracy_score(Y_train,Z):.2%}")
```

[88] ✓ 0.0s Python

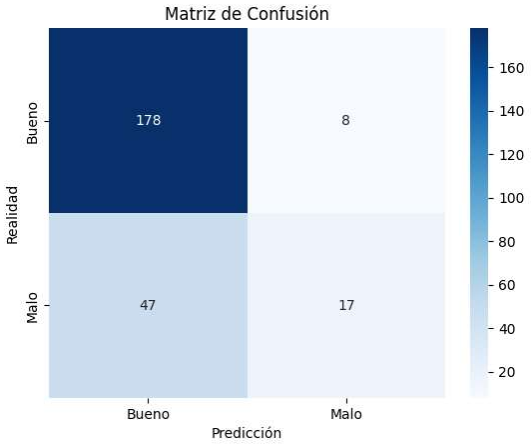
```
Precisión en entrenamiento: 84.27%
Precisión en prueba: 78.00%
```

Tras realizar varios cambios en los parámetros se llegó a un mejor resultado, en donde en el entrenamiento de alcanzó una exactitud del 84.27% y en la prueba un 78.00%. Los valores tienen una diferencia de aproximadamente 6%, y sin suficientemente elevados cómo para considerar que no hay underfiting.

Ahora, de forma análoga, se realiza el análisis de los resultados mediante la matriz de confusión.

```python
mtrx=confusion_matrix(Y_test,Z)

sb.heatmap(mtrx, annot=True, fmt="d", cmap="Blues",
           xticklabels=['Bueno', 'Malo'],
           yticklabels=['Bueno', 'Malo'])
plt.xlabel('Predicción')
plt.ylabel('Realidad')
plt.title('Matriz de Confusión')
plt.show()
```

[89] ✓ 0.4s Python



De aquí se obtienen de la misma forma los cálculos que se realizaron con la matriz de random forest:

	Procentajes
Exactitud	78.00%
Prevalencia	90.00%
Sensibilidad	95.70%
Especificidad	26.56%
Precisión	79.11%

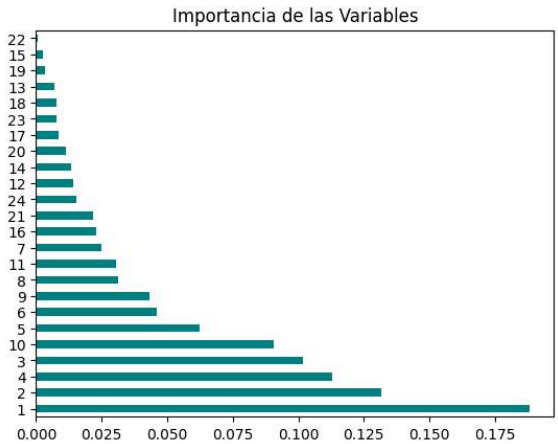
Cómo se puede apreciar, en términos generales, Gradient Boost tiene una exactitud y precisión ligeramente menores que en RandomForest, se podría considerar despreciable, sin embargo, si nos enfocamos en la especificidad, en Gradient Boost hay mucho más riesgo que en Random Fortes, GB es aún más ciego ante los riesgos, por lo que se puede considerar que de entre los dos modelos, el mejor si nos enfocamos en cada uno de los porcentajes, Random Forest resulta considerablemente mejor pese a su 37.5% de especificidad.

Conclusiones.

Para saber qué variables aportan más información, se puede usar los la función "`feature_importances_`" tanto en random forest cómo en gradient boosting.

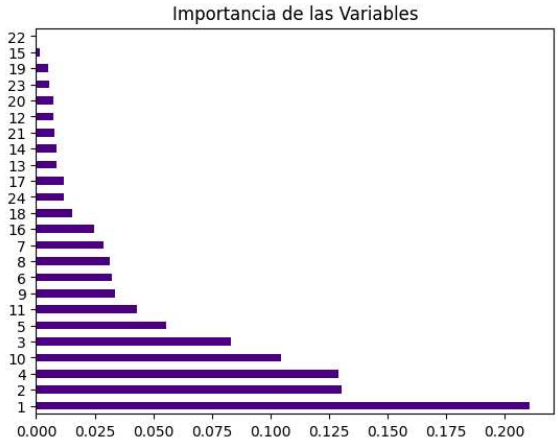
```python
importancias = ranForest.feature_importances_
nombres_columnas = X.columns

feat_imp = pd.Series(importancias, index=nombres_columnas).sort_values(ascending=False)
feat_imp.plot(kind='barh', color='teal')
plt.title("Importancia de las Variables")
plt.show()
```

[96] ✓ 0.2s Python




```python
importancias = gradBoost.feature_importances_

feat_imp = pd.Series(importancias, index=nombres_columnas).sort_values(ascending=False)
feat_imp.plot(kind='barh', color='indigo')
plt.title("Importancia de las Variables")
plt.show()
```

[102] ✓ 0.1s Python



En ambas gráficas se observa que las variables que más información aportan son **"1,2,4,10 y 3",** sin embargo, esto corresponde a la tabla "german.data-numeric", su equivalencia en la tabla de "german.data" son:

- Status of existing checking account (Cuenta corriente)
- Duration in month (Duración en meses)
- Credit amount (Monto de crédito)
- Credit history (Historial de crédito)
- Age (Edad)

Nuevamente, si vemos los porcentajes obtenidos en las matrices de confusión, se obtiene la tabla:

	Random Forest	Gradient Boosting
Exactitud	80.80%	78.00%
Prevalencia	87.20%	90.00%
Sensibilidad	95.70%	95.70%
Especificidad	37.50%	26.56%
Precisión	81.65%	79.11%

En general el rendimiento de ambos modelos es muy similar, difieren en porcentajes muy bajos, a excepción de la **especificidad**, donde Random Forest tiene aproximadamente un porcentaje del **11%** sobre Gradient Boosting. Como en el contexto del préstamo crediticio es importante disminuir los riesgos de concederle un préstamo a un cliente que potencialmente no cumplirá con el pago, es muy relevante la especificidad, nos dice el porcentaje de malos clientes que fueron detectados.

Tanto Random Forest cómo Gradient Boostitn tuvieron malos resultados en esta parte, sin embargo, el riesgo que corre Random Forest es considerablemente menor, por ello se puede decir que resultó para este contexto ser mejor modelo de aprendizaje.

Para mejorar posiblemente el cálculo de otorgamiento de créditos consideraría cómo variable

Tasa de desempleo regional	**2 : Tasa baja de desempleo** **1 : Tasa media de** **0: Tasa alta de desempleo**

Referencias

CRiskCo. (2026, 16 de febrero). *Paso a paso: el proceso de otorgamiento de crédito en México*. https://criskco.com/blog/proceso-otorgamiento-credito

Grus, J. (2019). *Data science from scratch: First principles with python* (2a ed.). O'Reilly Media.

VanderPlas, J. (2016). *Python Data Science Handbook*. O'Reilly Media.

Géron, A. (2019). *Hands-on machine learning with scikit-learn, keras, and tensorflow: Concepts, tools, and techniques to build intelligent systems (2nd edition)*. O'Reilly Media.